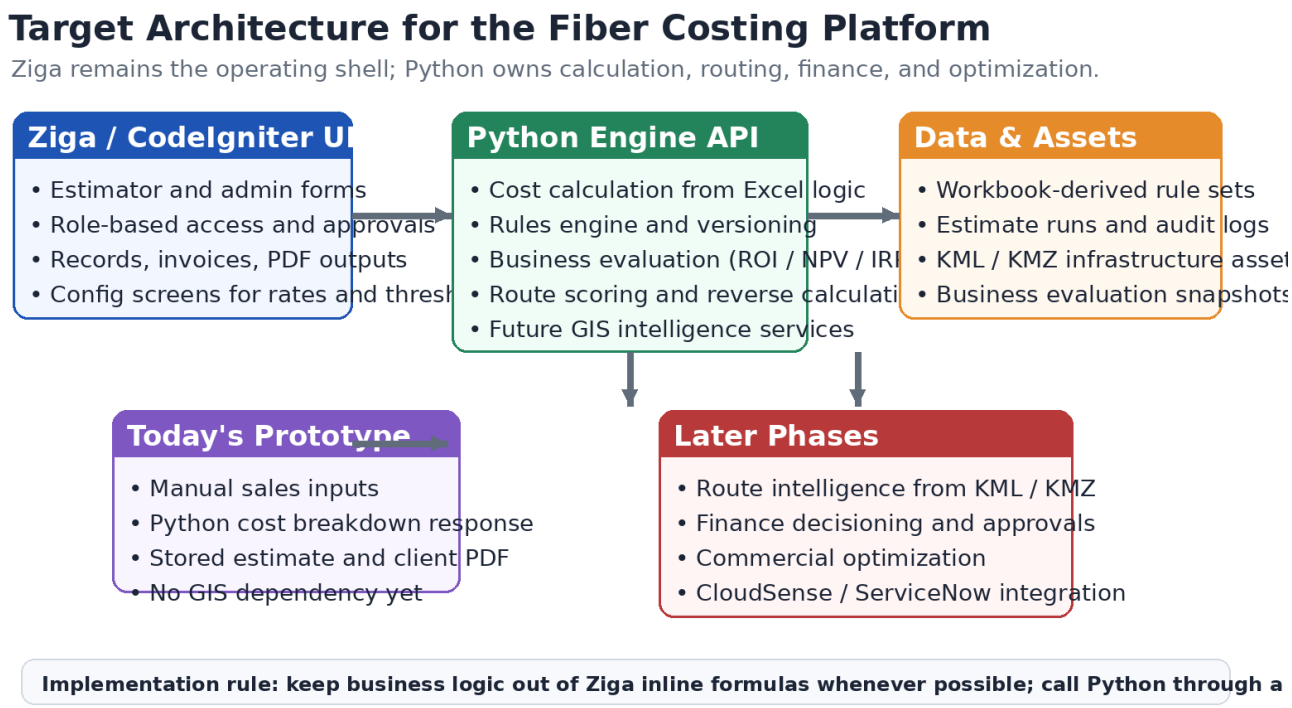
Fiber Costing Platform Execution Plan

Codex-ready roadmap for ZigaForm + Python delivery

Prepared for Client prototype and proposal planning	**Basis of plan** • Source inspection of ZigaForm CodeIgniter modules • Workbook review of the current cost model • Client meeting transcripts and future operating needs



Implementation rule: keep business logic out of Ziga inline formulas whenever possible; call Python through a

Recommendation in one line: keep Ziga as the operational shell, but move costing, rules, routing, finance, and optimization into Python services from the first prototype onward.

1. Executive summary

This project should be executed as a phased internal product, not as a one-off calculator. The workbook already behaves like a rule engine, and the Ziga source already provides a modular shell for forms, records, invoices, and add-ons. The correct architecture is therefore a hybrid: Ziga for UI, roles, records, and admin screens; Python for all intelligence and calculation services.

The fastest client-usable prototype for today is a manual-input estimate flow. The user enters the sales inputs already present in the workbook, Ziga sends those inputs to a small Python calculation API, the API returns the cost breakdown, Ziga stores the estimate record, and the user receives a quote-style PDF summary. That delivers immediate business value without waiting for GIS and route intelligence.

Future phases should add rule management, finance evaluation, KML/KMZ-driven route intelligence, reverse calculation, approval workflow, and enterprise integrations. The plan below is written so a codex agent can start implementation with clear module boundaries and low-risk sequencing.

2. What the uploaded assets tell us

The workbook is not a casual spreadsheet. It contains structured sections for labour cost, materials cost, administrative cost, wayleave cost, equipment, VAT, total due, and return-on-investment calculations. It already mixes linear formulas, rounding logic, threshold logic, and sales inputs such as build type, customer NRC, customer MRC, and exchange rate.

In the sample row set reviewed, the workbook uses formulas such as pole counts based on distance, closures based on long-span thresholds, supervision as a percentage of labour plus materials, and total due with VAT. The sample values loaded in the workbook produce a build cost of roughly ZMW 136,771 and an ROI of roughly 4.10 years for the visible scenario. Those numbers make the workbook suitable for validation testing during the first implementation stage.

The Ziga source is a modular CodeIgniter application. The relevant observed areas include application/modules/formbuilder for forms, records, invoices, and builder settings; application/modules/addon_webhook for outbound POST calls; and install/db/structure.sql for the current data model. The existing code already stores form definitions and records in JSON-heavy tables, which makes it feasible to extend without rewriting the shell.

3. Architecture decision

The architecture should be service-oriented from day one, even if it still feels like one app to users. Ziga remains the presentation and workflow layer. Python becomes the canonical execution layer for formula resolution, rule evaluation, business scoring, and all future GIS or optimization logic.

This matters because the client already wants more than cost arithmetic. The requested solution includes route recommendation, reasoned decisions, finance evaluation, and reverse solving. Those requirements are better served by Python libraries and more maintainable service code than by deep embedding inside Ziga form expressions.

Admins should be able to edit safe parameters and business rules through Ziga, but the underlying algorithmic engine must still live in Python. In practice, that means editable rates,

thresholds, weights, and flags in Ziga; protected execution logic in Python; and explicit versioning of every ruleset used for an estimate.

4. Confirmed Ziga touchpoints for the codex agent

These paths were directly observed in the source and are the safest starting points for extension work:

- application/modules/formbuilder/controllers/frontend.php — current front-end form flow, summaries, invoice rendering, and calculation-related handling.
- application/modules/formbuilder/views/forms/settings_form_calc.php — current builder-side calculation settings UI.
- application/modules/formbuilder/views/forms/formhtml_form.php — current form render path, hidden calculation variables, optional injected JavaScript, and front-end calculation bootstrapping.
- application/modules/formbuilder/models/model_forms.php — form metadata and JSON storage retrieval.
- application/modules/formbuilder/models/model_record.php — estimate / record retrieval including total amount and stored submission payloads.
- application/modules/addon_webhook/controllers/zfad_webhook_front.php — existing post-submit outbound webhook mechanism, useful as reference but not enough for real-time estimate calls.
- install/db/structure.sql — current table inventory, including cest_uiform_form, cest_uiform_form_records, cest_addon, and cest_addon_details.

Important design note: the existing webhook add-on fires after submission. It is helpful as a reference for transport and authentication patterns, but it is not by itself the best mechanism for synchronous quote calculations. For the prototype, the codex agent should add a dedicated AJAX endpoint or lightweight integration layer so the user can see results before final submission.

5. Phase 1 — today’s working prototype

Target outcome for today: a client-usable quote prototype running inside Ziga, using Python for the real calculation, and able to save estimate records plus generate a PDF output.

5.1 Scope of the prototype

- Manual input only. No GIS, no route generation, no KML/KMZ ingest yet.
- Sales-input driven flow matching the visible workbook inputs: distance split fields as needed, build option fields, customer NRC, customer MRC, exchange rate, and any directly required override values.
- Python endpoint returns a structured cost breakdown: labour, materials, administrative cost, wayleave, equipment, subtotal, VAT, total due, build cost, net build cost, and ROI metrics.
- Ziga shows the estimate summary, stores the record, and produces a client-facing PDF or invoice-style summary.
- Admin can edit rates and thresholds outside code where practical, but only for the narrow prototype dataset.

5.2 What the codex agent should build today

Workstream	Deliverable	Main touchpoints	Output by end of day
Ziga form shell	One estimator form with clean sections for sales inputs, estimate summary, and submit/save flow	formbuilder form definitions; formhtml_form.php; optional custom JS	Usable screen for client demo
Python calc service	Small API with one /estimate endpoint and workbook-matched formulas	new Python service folder outside Ziga web root	Canonical quote computation result in JSON
Ziga integration layer	AJAX bridge from form to Python and response mapping back into visible fields / hidden fields	new controller or custom endpoint; frontend.php integration	Live calculation inside Ziga without final submit
Persistence	Store submitted estimate plus engine response snapshot and ruleset version	model_record extension and/or new custom table	Audit-ready prototype records
PDF output	Quote / estimate PDF from saved record	existing invoice/summary templates as starting point	Client can leave with printable estimate

5.3 Recommended prototype flow

1. User opens the estimator form in Ziga.
2. User enters the required sales inputs for the quote.
3. Front-end JavaScript posts a normalized payload to a new Ziga controller endpoint, or directly to the Python service through a controlled bridge.
4. Python runs the workbook-derived formulas and returns a versioned JSON response with both totals and breakdown lines.
5. Ziga renders the result sections and unlocks the save / generate-PDF action.
6. On save, Ziga persists the original input payload, the engine response snapshot, the active ruleset version, and the generated total.
7. Ziga renders a client-facing PDF summary using its existing record/invoice template flow as the base.

5.4 Prototype data contract

The codex agent should keep the first API contract narrow and stable. A suggested request and response shape is below.

Request payload	Response payload
estimate_id (optional draft id) rule_set_version build_profile distance_new_aerial distance_new_underground distance_existing_aerial distance_existing_duct customer_nrc customer_mrc exchange_rate manual_overrides (optional)	line_items by category labour_total materials_total admin_total wayleave_total equipment_total subtotal vat total_due build_cost net_build_cost roi_months roi_years warnings engine_version calculated_at

The key is that Ziga stores the full response snapshot instead of trying to reconstruct the calculation later. That preserves auditability and makes future engine changes safe.

5.5 Codex task list for today

Task A — stand up the Python service

- Create a dedicated service directory with a minimal FastAPI or Flask app.
- Implement one estimate endpoint and one health endpoint.
- Encode only the workbook formulas needed for the first prototype.
- Include unit tests that reproduce the visible sample workbook case and compare expected totals.
- Return structured JSON, not only one total number.

Task B — create the first estimator form in Ziga

- Use the formbuilder module to create the visible sales-input sections.
- Keep the UX short: client details, quote assumptions, estimate output, actions.
- Hide internal engine fields from normal users.
- Use Ziga only for capture and display, not as the source of truth for complex formulas.

Task C — add real-time estimate call

- Implement a custom controller endpoint in Ziga that forwards validated payloads to the Python service.
- Add front-end JavaScript to trigger estimate calculation on a button click or controlled field changes.
- Map response fields back into the visible summary panel and hidden persistence fields.
- Gracefully show warnings and validation errors.

Task D — persistence and versioning

- Save the input payload, engine response, and engine version with each record.
- Prefer a dedicated custom table for engine snapshots if the built-in record table becomes too cramped.
- Never save only the displayed total; save the breakdown and metadata too.

Task E — client PDF output

- Reuse the existing invoice/summary rendering path as a base.
- Rename the output from invoice language to estimate / quotation language.
- Include the summary inputs, category totals, VAT, final due, and ROI summary.
- Add a disclaimer that this is a prototype estimate subject to technical validation.

5.6 Suggested prototype persistence model

The existing tables can handle forms and records, but the intelligence layer needs cleaner version tracking. The codex agent should add the following custom tables rather than overloading every concern into the generic form record table.

Table	Purpose	Key fields
fiber_rule_set	Named and versioned set of editable rates and thresholds	id, code, version, status, data_json, created_by, activated_at
fiber_engine_run	Immutable snapshot of each engine response	id, record_id, rule_set_id, request_json, response_json, engine_version, duration_ms
fiber_quote_meta	Quote-level metadata and business status	id, record_id, quote_number, client_name, status, pdf_path

6. Future phases for the business proposal

Phase	Main goal	What gets built	Business outcome
Phase 2 — rules admin hardening	Make the prototype maintainable	Full admin UI for rates, rule sets, thresholds, build profiles, activation/deactivation, and change logs	Business users can maintain pricing assumptions without code deployments
Phase 3 — business evaluation engine	Move from quote to decision support	ROI, NPV, IRR, payback logic, threshold checks, pass/fail statuses, business-case report	Commercial team can decide whether an opportunity should move forward

Phase 4 — GIS and route intelligence	Reduce manual survey dependence	KML/KMZ import, asset normalization, nearest-joint logic, route candidates, cost-aware recommendation notes	Technical team gets faster and more defensible estimate paths
Phase 5 — reverse calculation and optimization	Support target-driven pricing	Solve for MRC, NRC, payback, or target ROI under current constraints; compare scenarios	Sales team can negotiate based on targets rather than guesswork
Phase 6 — approval workflow	Operationalize trust and governance	Technical review, finance approval, locked result snapshots, comments, audit trail, unique business-evaluation IDs	Approvals become traceable and easier to defend
Phase 7 — enterprise integration	Connect the app into delivery systems	Exports and APIs for CloudSense/Salesforce, ServiceNow, notifications, and downstream handoff	Removes duplicate entry and shortens order-to-implementation time

7. Ownership matrix: what lives in Ziga vs Python

Concern	Primary owner	Reason
Estimator UI, forms, and record screens	Ziga	This is already what the product shell does well.
Admin editing of rates, thresholds, and rule versions	Ziga + Python-backed tables	Users need editable configuration, but Python still executes the rules.
Workbook-derived formulas and totals	Python	This is the core business logic and should not be trapped in form expressions.
Business evaluation metrics	Python	These calculations will grow in complexity and need testability.

GIS routing and recommendation logic	Python	Geospatial and optimization libraries belong in Python.
PDF generation and quote presentation	Ziga initially	The current invoice/summary flow is a fast starting point.
Audit trail and approvals	Ziga with engine snapshots	Workflow sits near users; calculation evidence comes from Python.
Future front-end replacement	Future custom UI	Clean APIs mean the engine can survive a front-end change.

8. Risks, constraints, and control measures

- The workbook almost certainly contains hidden assumptions and magic constants. Control: convert formulas into named rule blocks, not anonymous code copied from cells.
- Ziga already has its own front-end calculation features. Risk: duplicated logic between JavaScript formulas and Python. Control: use Python as the canonical engine and keep front-end calculations cosmetic only.
- The existing webhook add-on is asynchronous and post-submit oriented. Risk: poor UX if reused directly. Control: add a synchronous estimate endpoint for pre-submit calculation.
- Admin-editable rules can become dangerous if they include raw formulas. Control: expose only safe parameters and structured rule settings in early phases.
- GIS data quality will vary in rural areas. Control: keep route intelligence as a later phase with explicit fallback modes and manual overrides.

9. Proposal framing points you can reuse

- The product starts by automating the current workbook-based costing process, reducing manual effort and increasing consistency.
- The architecture is intentionally phased so the business gets a usable prototype immediately while protecting the long-term path to GIS intelligence and enterprise integration.
- The solution keeps users in one operational front end while placing the core decision intelligence in a maintainable Python service layer.
- The platform will mature from quote generation into a full technical-commercial decision engine with approval and integration capability.

10. Prototype acceptance criteria for today

- The client can open one estimator form and produce a quote using manual inputs only.
- The returned totals match the workbook logic for the implemented scope within an agreed tolerance.

- The estimate can be saved as a record and reopened.
- A client-facing PDF can be generated from the saved estimate.
- The engine response and ruleset version are persisted for later review.
- The prototype clearly labels deferred features such as GIS routing and finance evaluation beyond the implemented scope.

11. Final recommendation

Do not treat today’s prototype as a disposable demo. Build it as the first thin slice of the final architecture: Ziga for user flow and operations, Python for the intelligence core, versioned rule sets for safety, and persistent engine snapshots for auditability. If the codex agent follows that structure now, the same base can support the client demo, the proposal, and the next development phases without a major rewrite.